

Data Structures

Interview Cracker question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

Arrays and Strings

The most common interview substrate. Most "easy" and "medium" problems are an array or a string with a pattern applied to it.

[Easy] Q. What is the difference between an array and a dynamic array?

Answer. A **static array** is a fixed-size, contiguous block of memory. Its size is decided at creation and never changes. `int[10]` in C or Java.

A **dynamic array** wraps a static array and grows it when it fills up, by allocating a larger block and copying. Python `list`, Java `ArrayList`, C++ `vector`, Go `slice`, JavaScript `Array`.

Both give $O(1)$ indexed access, because the address of element `i` is just `base + i * element_size - one` multiplication and one addition, no traversal.

Operation	Cost	Why
Access by index	$O(1)$	Direct address arithmetic
Append	$O(1)$ amortised	Occasional doubling + copy
Insert/delete at front or middle	$O(n)$	Everything after must shift
Search (unsorted)	$O(n)$	No structure to exploit
Search (sorted)	$O(\log n)$	Binary search

[Easy] Q. Why is inserting at the front of an array $O(n)$ but appending $O(1)$?

Answer. Because arrays are contiguous. Inserting at index 0 means every existing element must move one slot to the right to make room - n moves. Appending writes into already-reserved space at the end - one move.

This is the single most useful performance fact about arrays, and it is why:

- `list.pop(0)` is a performance bug in a loop; use `collections.deque`.
 - Building a result by `insert(0, x)` is $O(n^2)$; append and reverse at the end.
-

[Medium] Q. Why are strings immutable in Python, Java and JavaScript?

Answer. Several reasons reinforce each other:

- **Safe sharing.** An immutable string can be passed around, cached and shared
- **Hashability.** A string's hash can be computed once and cached. If strings
- **Interning.** Identical literals can share one object, saving memory.
- **Security.** A filename or URL validated and then passed to the OS cannot be

The cost is that every "modification" allocates. Concatenating in a loop is $O(n^2)$; use a builder (`StringBuilder`, `"".join`, `array + join`) instead.

[Medium] Q. What does "in-place" mean, and what is the space complexity of an in-place algorithm?

Answer. In-place means the algorithm transforms the input using only $O(1)$ extra space - a few variables, no second array proportional to n .

```
def reverse_in_place(arr):          # O(1) extra space
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
```

Two honest caveats interviewers probe for:

- You cannot do this to a string in Python/Java - strings are immutable, so you
- $O(\log n)$ stack space from recursion is usually still called "in-place" by

[Medium] Q. What is the two-pointer technique and when does it apply?

Answer. Two indices moving through the array under a rule, replacing a nested loop - turning $O(n^2)$ into $O(n)$.

Three shapes:

Converging (from both ends) - needs a **sorted** array or a symmetry:

```
left, right = 0, len(arr) - 1
while left < right:
    total = arr[left] + arr[right]
    if total == target: return (left, right)
    if total < target: left += 1      # need bigger
    else:             right -= 1     # need smaller
```

This works only because sorting guarantees which direction increases the sum.

Fast/slow - cycle detection, finding the middle of a linked list.

Same direction (read/write) - in-place filtering, deduplication:

```
write = 0
for read in range(len(arr)):
    if keep(arr[read]):
        arr[write] = arr[read]
        write += 1
return write          # new logical length
```

The tell: the array is sorted, or you are looking for a pair/triplet, or you must filter in place.

[Medium] Q. What is the sliding window technique?

Answer. A window `[left, right]` over a contiguous run, where `right` expands and `left` contracts to restore a constraint. Each element enters and leaves at most once, so the whole scan is $O(n)$ despite the nested-looking loops.

```
def longest_unique(s):
    last_seen = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1      # contract past the duplicate
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

The tell: "longest/shortest/maximum **contiguous** subarray or substring satisfying X". If the problem says subsequence (non-contiguous), a window does not apply - that is usually dynamic programming.

[Medium] Q. How do you find a duplicate in an array? Give three approaches.

Answer. The interviewer wants to see you trade time against space.

- 1 **Brute force** - compare every pair. $O(n^2)$ time, $O(1)$ space.
- 2 **Sort first** - duplicates become adjacent. $O(n \log n)$ time, $O(1)$ extra space
- 3 **Hash set** - one pass, check membership. $O(n)$ time, $O(n)$ space.

Then the constrained follow-up: values are in $1 \dots n$ and you may not modify the input or use extra space. That is Floyd's cycle detection applied to the array as an implicit linked list ($i \rightarrow \text{arr}[i]$), giving $O(n)$ time and $O(1)$ space.

Say all three and name the trade-off explicitly - that is the answer they want, not just the fastest one.

[Easy] Q. What is the difference between a subarray, a subsequence and a subset?

Answer. A distinction that changes which algorithm applies, so get it right:

Term	Contiguous?	Order kept?	Count for n elements
Subarray / substring	Yes	Yes	$n(n+1)/2$
Subsequence	No	Yes	2^n
Subset	No	No	2^n

For $[1, 2, 3]$: $[2, 3]$ is a subarray; $[1, 3]$ is a subsequence but not a subarray; $\{3, 1\}$ is a subset.

Why it matters: subarray problems usually yield to a sliding window or prefix sums ($O(n)$). Subsequence problems usually need dynamic programming, because there are exponentially many.

[Medium] Q. What is a prefix sum and what is it for?

Answer. A precomputed array where `prefix[i]` is the sum of the first i elements. It makes any range sum $O(1)$:

```
prefix = [0]
for n in nums:
    prefix.append(prefix[-1] + n)

# sum of nums[i:j] in O(1)
range_sum = prefix[j] - prefix[i]
```

Build cost $O(n)$ once, then unlimited $O(1)$ range queries - the right trade when you answer many queries over a static array. Combined with a hash table of seen prefix sums it solves "count subarrays summing to k" in one pass.

Worked examples: [coding-challenges/two-pointers/](#) and [coding-challenges/sliding-window/](#).

Hash Tables

The most useful data structure in interviews. A very large fraction of "optimise this $O(n^2)$ solution" problems are solved by adding a hash table.

[Easy] Q. How does a hash table work?

Answer. It stores key-value pairs in an array, and uses a hash function to compute which slot a key belongs in.

- 1 `hash(key)` produces an integer.
- 2 That integer is reduced to an index, usually `hash % capacity`.
- 3 The pair is stored at that index.

Lookup repeats the same computation, so it goes straight to the slot instead of scanning - $O(1)$ average instead of $O(n)$.

The whole structure rests on one assumption: that the hash function spreads keys roughly evenly. Everything that goes wrong with hash tables is that assumption failing.

[Easy] Q. What is a collision, and how are collisions resolved?

Answer. A collision is two different keys mapping to the same slot. They are unavoidable - there are more possible keys than slots (the pigeonhole principle).

Two families of solution:

Separate chaining. Each slot holds a list of entries. On collision, append. Lookup scans that one short list. Simple, degrades gracefully, but costs a pointer per entry and scatters memory. Used by Java's `HashMap` (which upgrades a long chain to a balanced tree) and most C++ `unordered_map` implementations.

Open addressing. Everything lives in the array itself. On collision, probe for another free slot - linearly, quadratically, or via a second hash. Better cache behaviour, no per-entry allocation, but deletion is awkward (you must leave a tombstone, or later probes stop too early) and it degrades badly when full. Used by Python's `dict` and Go's maps.

[Medium] Q. Hash table lookup is " $O(1)$ ". When is it not?

Answer. $O(1)$ is the average case. The worst case is $O(n)$, when every key hashes to the same slot and the table degenerates into a linked list.

This matters in practice in two ways:

- **Bad hash functions.** If you write `hashCode()` as `return 1`, every lookup
- **Hash-flooding attacks.** If an attacker can choose your keys and knows your

Java 8+ mitigates it differently: a bucket whose chain exceeds 8 entries converts to a red-black tree, capping the worst case at $O(\log n)$.

[Medium] Q. What is the load factor and why does it trigger a resize?

Answer. Load factor = entries / capacity. It measures how full the table is.

As it rises, collisions rise, and average probe length rises with it. Past roughly 0.7 the performance falls off sharply, so implementations resize - double the capacity and **rehash every entry**, because the index depends on capacity.

That rehash is $O(n)$, which makes a single insert occasionally $O(n)$ - but amortised $O(1)$ across all inserts, for exactly the same geometric-growth reason a dynamic array append is.

Practical consequence: if you know you will insert a million entries, pre-size the map. You avoid about 20 rehashes.

[Medium] Q. What makes a good hash function?

Answer. Three properties:

- 1 **Deterministic** - the same key always hashes to the same value. Within one
- 2 **Uniform** - outputs spread evenly across the range, so no slot is favoured.
- 3 **Fast** - it runs on every single operation.

Note that cryptographic strength is not required and usually not wanted; SHA-256 is uniform but far too slow for a hash table.

[Hard] Q. What is the contract between equals and hashCode?

Answer. If two objects are equal, they must have the same hash code. The converse is not required - unequal objects may share a hash code, that is just a collision.

Break it and hash tables break silently:

```
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));
set.contains(new Point(1, 2)); // false, if hashCode() was not overridden
```

The lookup hashes to a different slot and never finds the entry, even though `equals` would have said yes.

The other half of the contract: **a key's hash must not change while it is in the table**. Mutating a field that feeds `hashCode` strands the entry in the wrong slot - it is in the table but unreachable, and it will not even be removed by `remove()`. This is why immutable keys are strongly preferred, and why Python simply refuses to hash a `list`.

[Medium] Q. When would you not use a hash table?

Answer. Four cases:

- **You need ordering.** Hash tables have none. Use a balanced BST (`TreeMap`,
 - **You need the worst case bounded.** Real-time systems cannot accept an
 - **The dataset is tiny.** For five entries, a linear scan of an array is faster
 - **Keys are not hashable.** Mutable or unorderable keys.
-

[Medium] Q. What is the difference between a HashMap, a HashSet and a HashTable?

Answer.

- **Map** - key -> value. The core structure.
 - **Set** - keys only. Almost always implemented as a map with a dummy value.
 - **Hashtable** (the legacy Java class, capital-T) - a synchronised, slower
-

[Medium] Q. How do you use a hash table to turn $O(n^2)$ into $O(n)$?

Answer. The pattern: **replace an inner search loop with a lookup**, trading memory for time.

Two Sum is the canonical example. The brute force checks every pair:

```
for i in range(len(nums)):          # O(n^2)
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            return [i, j]
```

The hash table version asks, for each number, "have I already seen the value that completes this pair?" - a lookup instead of a scan:

```
seen = {}                          # value -> index
for i, n in enumerate(nums):        # O(n) time, O(n) space
    if target - n in seen:
        return [seen[target - n], i]
    seen[n] = i
```

Recognising this - "I am searching inside a loop, so I should index instead" - is worth more than memorising any individual solution. Worked examples are in [coding-challenges/hashing/](#).

Linked Lists

Rarely the right choice in production, reliably popular in interviews - because they test pointer manipulation and careful edge-case handling with no library to hide behind.

[Easy] Q. What is a linked list and how does it differ from an array?

Answer. A chain of nodes, each holding a value and a reference to the next node. Nodes live anywhere in memory; the links impose the order.

	Array	Linked list
Access by index	O(1)	O(n) - must walk
Insert/delete at head	O(n)	O(1)
Insert/delete at a known node	O(n)	O(1)
Memory per element	value only	value + pointer(s)
Cache locality	Excellent	Poor
Size	grows by reallocation	grows one node at a time

The honest summary: linked lists win only when you are inserting or deleting at a position you already hold a reference to, and never index. In practice, cache locality means an array often wins even then - a linear scan of a contiguous array can outrun pointer-chasing by an order of magnitude.

[Easy] Q. Singly vs doubly linked - what does the extra pointer buy?

Answer. A `prev` pointer costs one word per node and buys:

- **Backwards traversal.**
- **O(1) deletion given only the node** - a singly linked list needs the
- **O(1) append with a tail pointer** - also possible singly, but removing the

This is why an LRU cache uses a doubly linked list: it must move an arbitrary node to the front in O(1), which requires unlinking it, which requires `prev`.

[Medium] Q. How do you detect a cycle in a linked list?

Answer. Floyd's tortoise and hare. Two pointers, one moving one step per iteration and one moving two. If there is a cycle they must eventually meet; if there is not, the fast pointer reaches the end.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

O(n) time, O(1) space. The hash-set alternative (store every visited node) is also O(n) time but O(n) space - mention both, then say why you would pick Floyd.

Why they must meet: once both are inside the cycle, the fast pointer gains exactly one position per iteration, so it closes any gap of size k in k steps. It cannot "jump over" the slow pointer.

The follow-up - find where the cycle starts. After they meet, reset one pointer to the head and advance both one step at a time. They meet at the cycle entrance. (The distance from head to entrance equals the distance from the meeting point to the entrance, going forward around the loop.)

[Medium] Q. Reverse a singly linked list.

Answer. Three pointers, walking once and flipping each link as you pass.

```
def reverse(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next    # save it BEFORE overwriting
        curr.next = prev        # flip
        prev = curr              # advance
        curr = next_node
    return prev                  # new head
```

O(n) time, O(1) space. The whole problem is saving `curr.next` before you destroy it - do that in the wrong order and you lose the rest of the list.

The recursive version is elegant but **O(n) space** from the call stack, which is a real downside worth naming rather than presenting it as equally good.

[Medium] Q. Find the middle of a linked list in one pass.

Answer. Same fast/slow trick. When the fast pointer reaches the end, the slow pointer is at the middle.

```
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

For even lengths this returns the second of the two middles. If you need the first, stop when `fast.next` and `fast.next.next`. Interviewers often care which one you return - say out loud which convention you chose.

[Medium] Q. Why does so much linked-list code use a dummy head node?

Answer. Because it removes the "what if the head changes?" special case.

Without a dummy, deleting the first node or inserting before it needs separate branching, and that branch is where the bug always is. With a sentinel node in front, every node has a predecessor and one uniform code path handles all of them:

```
def remove_all(head, target):
    dummy = Node(None)
    dummy.next = head
    prev = dummy
    while prev.next:
        if prev.next.value == target:
            prev.next = prev.next.next    # works even for the real head
        else:
            prev = prev.next
    return dummy.next                    # not `head` - it may be gone
```

Reaching for a dummy node unprompted signals real experience.

[Hard] Q. Merge two sorted linked lists. What is the complexity, and why is it nicer than for arrays?

Answer. Walk both with a pointer each, always taking the smaller head, and splice it onto a result list built behind a dummy node.

$O(n + m)$ time, $O(1)$ extra space - and that $O(1)$ is the interesting part. The array version of merge needs an $O(n + m)$ output buffer, because you cannot make room in a contiguous block without copying. A linked list merge just re-points existing nodes, allocating nothing.

This is the one place linked lists genuinely beat arrays, and it is why merge sort on a linked list is $O(1)$ -space while merge sort on an array is $O(n)$ -space.

A worked implementation is in `examples/java/algorithms/merge_two_sorted_lists.java` and `examples/c/data-structures/linked_list.c`.

[Medium] Q. What edge cases must linked-list code handle?

Answer. Interviewers grade heavily on these, and they are the same four every time:

- 1 **Empty list** (`head` is `None`).
- 2 **Single node** (`head.next` is `None`) - many loops never execute.
- 3 **Operating on the head** - does your code return the right new head?
- 4 **Operating on the tail** - is `next` set to `None`, or did you leave a

Walk through a 0-node, 1-node and 2-node list out loud before you say you are done. It takes twenty seconds and catches most bugs.

Stacks, Queues and Heaps

Three small structures that unlock a surprising number of problems once you learn to recognise their signals.

[Easy] Q. Stack vs queue?

Answer. Both restrict where you may add and remove; they differ in which end.

- **Stack** - LIFO, last in first out. `push/pop` at the same end.
- **Queue** - FIFO, first in first out. Add at the back, remove from the front.

All four operations are $O(1)$ in a good implementation.

Where each shows up:

Stack	Queue
Function call stack	BFS / level-order traversal
Undo history	Task and message queues
Bracket matching	Print spoolers, request buffers
DFS (explicit or via recursion)	Rate limiting, buffering
Expression evaluation	Producer-consumer pipelines

The rule of thumb: DFS uses a stack, BFS uses a queue. That one fact decides a lot of problems.

[Easy] Q. How do you implement a queue efficiently?

Answer. Not with a plain array and `pop(0)` - that shifts every element, so it is $O(n)$ per dequeue and $O(n^2)$ overall.

Three correct options:

- **Doubly linked list** - $O(1)$ at both ends, but a pointer per node and poor
- **Circular buffer** - a fixed array with `head` and `tail` indices that wrap via
- **Deque from the standard library** - `collections.deque`, `ArrayDeque`,

The classic puzzle - implement a queue with two stacks - is worth knowing: push onto `in`; to pop, if `out` is empty pour all of `in` into it (reversing order) then pop from `out`. Each element moves at most twice, so it is **$O(1)$ amortised**.

[Medium] Q. What is a monotonic stack and what does it solve?

Answer. A stack kept in sorted order - you pop elements that break the order before pushing. It answers "for each element, what is the next element greater (or smaller) than it?" in **$O(n)$** instead of $O(n^2)$.

```
def next_greater(nums):
    result = [-1] * len(nums)
    stack = []  # holds indices, values decreasing
    for i, n in enumerate(nums):
        while stack and nums[stack[-1]] < n:
            result[stack.pop()] = n  # n is the answer for that index
        stack.append(i)
    return result
# [2, 1, 3] -> [3, 3, -1]
```

Each index is pushed once and popped once, so despite the inner `while` the total work is $O(n)$.

The tell: "next greater / previous smaller element", "largest rectangle in a histogram", "daily temperatures", "trapping rain water". If a problem asks you to relate each element to the nearest element satisfying a comparison, try a monotonic stack.

[Easy] Q. What is a heap, and what is it good at?

Answer. A complete binary tree satisfying the heap property: in a min-heap, every parent is $\leq$ its children (so the minimum sits at the root).

It is stored in a **flat array**, not with pointers - for index i , children are at $2i+1$ and $2i+2$, parent at $(i-1) // 2$. That is why heaps are fast: no allocation, excellent cache locality.

Operation	Cost
Peek min/max	$O(1)$
Insert	$O(\log n)$ - sift up
Extract min/max	$O(\log n)$ - sift down
Build from n items	$O(n)$, not $O(n \log n)$
Search for arbitrary value	$O(n)$ - no ordering between siblings

Two points interviewers probe:

- **Building a heap is $O(n)$.** Sifting down from the middle backwards costs less
 - **A heap is only partially ordered.** Finding an arbitrary element is $O(n)$, and
-

[Medium] Q. How do you find the k largest elements in a stream of n items?

Answer. A min-heap of size k .

```
import heapq

def k_largest(stream, k):
    heap = []                                # min-heap of the best k so far
    for item in stream:
        if len(heap) < k:
            heapq.heappush(heap, item)
        elif item > heap[0]:                 # bigger than the smallest kept
            heapq.heapreplace(heap, item)
    return sorted(heap, reverse=True)
```

$O(n \log k)$ time, $O(k)$ space.

The counter-intuitive part, and the point of the question: to track the k largest you use a **min**-heap. The root is the weakest of your current champions, so it is both the cheapest to check against and the right one to evict.

Why not sort? Sorting is $O(n \log n)$ time and **$O(n)$ space** - impossible if the stream does not fit in memory. When k is small and n is huge, $O(n \log k)$ with $O(k)$ space is the meaningful win.

[Medium] Q. What is a priority queue and how does it differ from a heap?

Answer. A **priority queue** is an abstract data type: "give me the highest-priority item next". A **heap** is the usual concrete implementation.

The distinction matters because other implementations exist with different trade-offs - a sorted array gives $O(1)$ extraction but $O(n)$ insertion; a Fibonacci heap gives $O(1)$ amortised insert and decrease-key, which is what makes the textbook Dijkstra bound $O(E + V \log V)$.

In interviews, "priority queue" almost always means "binary heap". It is the engine inside Dijkstra's algorithm, A*, Huffman coding, and event-driven simulation.

[Medium] Q. How do you check for balanced brackets?

Answer. The canonical stack problem. Push opening brackets; on a closing bracket, the top of the stack must be its partner.

```
PAIRS = {"(": ")", "[": "]", "{": "}"

def is_balanced(s):
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in PAIRS:
            if not stack or stack.pop() != PAIRS[ch]:
                return False          # wrong partner, or nothing open
    return not stack                  # anything left open is unbalanced
```

$O(n)$ time, $O(n)$ space. The two edge cases that separate a correct answer from a nearly-correct one: a closing bracket when the stack is **empty**, and leftover openers at the **end**. Candidates routinely forget the second and return `True` for `"((("`.

Trees and Graphs

The highest-yield topic after hash tables. Most "hard" interview problems are a graph problem wearing a costume.

[Easy] Q. What is a binary search tree, and what is its complexity?

Answer. A binary tree where, for every node, all keys in the left subtree are smaller and all keys in the right subtree are larger. That invariant makes search a series of "left or right?" decisions.

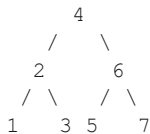
Search, insert and delete are **O(h)** where h is the height - **not** O(log n). The two coincide only when the tree is balanced:

- Balanced: $h \sim \log n \rightarrow \mathbf{O(\log n)}$.
- Degenerate (insert already-sorted data): $h = n \rightarrow \mathbf{O(n)}$, a linked list with

That degeneration is the entire reason self-balancing trees exist. Saying "O(h), which is O(log n) only if balanced" is a much stronger answer than "O(log n)".

[Easy] Q. What are the tree traversals and when do you use each?

Answer. Three depth-first orders, defined by when you visit the node relative to its children, plus one breadth-first order.

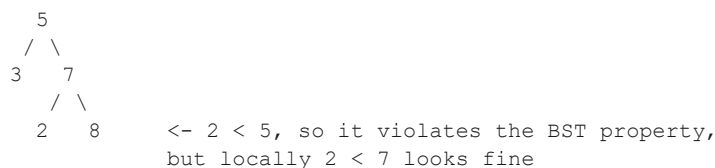


Traversal	Order	Result	Use
In-order	left, node , right	1 2 3 4 5 6 7	Sorted output from a BST
Pre-order	node , left, right	4 2 1 3 6 5 7	Copying/serialising a tree
Post-order	left, right, node	1 3 2 5 7 6 4	Deleting; computing from children up
Level-order (BFS)	by depth	4 2 6 1 3 5 7	Shortest path; level-by-level work

The fact worth memorising: **in-order traversal of a BST yields sorted order**. "Validate a BST" and "find the kth smallest" are both just that observation.

[Medium] Q. How do you validate a binary search tree?

Answer. The trap: checking only `left < node < right` locally is **wrong**. It passes trees that are not BSTs:



Every node must fall inside a range inherited from its ancestors:

```
def is_bst(node, low=float("-inf"), high=float("inf")):
```

```

if node is None:
    return True
if not (low < node.value < high):
    return False
return (is_bst(node.left, low, node.value) and
        is_bst(node.right, node.value, high))

```

Equivalently: do an in-order traversal and check the output is strictly increasing. Both are $O(n)$ time, $O(h)$ space.

[Medium] Q. Which balanced trees should I know, and what balances them?

Answer. You need the idea, not the rotation code.

- **AVL** - strictly balanced; heights of sibling subtrees differ by at most 1.
- **Red-black** - loosely balanced via colour rules; the longest path is at most
- **B-tree / B+ tree** - high branching factor, so the tree is very shallow. Used

All give $O(\log n)$ worst case. The distinction is the constant factor and the read/write balance.

[Easy] Q. BFS or DFS - how do you choose?

Answer.

	BFS	DFS
Structure	Queue	Stack (or recursion)
Visits	Nearest first	Deepest first
Finds shortest path	Yes , in an unweighted graph	No
Space	$O(\text{width})$ - can be huge	$O(\text{depth})$
Natural for	Levels, shortest hops, "minimum steps"	Cycles, topological sort, connectivity, backtracking

Both are $O(V + E)$ time.

The decisive rule: if the question says shortest, fewest, or minimum number of steps in an unweighted graph, use **BFS**. BFS reaches every node by the fewest edges, so the first time it arrives is the shortest way. DFS gives no such guarantee - it may find a long path first.

If edges are weighted, neither works: use **Dijkstra** (non-negative weights) or **Bellman-Ford** (negative weights allowed).

[Medium] Q. How do you represent a graph, and which representation should you pick?

Answer. Two options, chosen by density.

Adjacency list - map each vertex to its neighbours.

```
graph = {"a": ["b", "c"], "b": ["d"], "c": [], "d": []}
```

Space $O(V + E)$. Iterating a vertex's neighbours is $O(\text{degree})$. **The default.**

Adjacency matrix - a $V \times V$ grid where $m[i][j]$ marks an edge. Space $O(V^2)$ regardless of edge count. Edge lookup is $O(1)$; listing neighbours is $O(V)$.

Pick the matrix when the graph is **dense** (E approaches V^2), when you need constant-time "is there an edge?", or for matrix algorithms like Floyd-Warshall. Pick the list otherwise - real graphs are almost always sparse, and $O(V^2)$ space for a million vertices is a terabyte.

[Medium] Q. Why must graph traversal track visited nodes when tree traversal need not?

Answer. Because a tree is by definition acyclic and each node has exactly one parent - you cannot arrive anywhere twice. A general graph can have cycles and multiple paths to the same node, so without a `visited` set traversal loops forever or does exponential redundant work.

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    if node in visited:
        return
    visited.add(node)
    for neighbour in graph[node]:
        dfs(graph, neighbour, visited)
```

Mark a node **when you enqueue/visit it**, not when you dequeue it - otherwise the same node can be queued many times before it is ever processed.

[Hard] Q. What is a topological sort and when is it impossible?

Answer. A linear ordering of a directed graph's vertices such that every edge points forward - every task appears after everything it depends on. Build systems, task schedulers and course prerequisites all need it.

It is impossible exactly when the graph contains a cycle, since a cycle means a set of tasks that each depend on the other. So a topological sort doubles as a cycle detector, which is often the real question.

Kahn's algorithm, $O(V + E)$:

- 1 Compute every vertex's in-degree.
- 2 Enqueue all vertices with in-degree 0.
- 3 Pop one, append it to the output, and decrement its neighbours' in-degrees;
- 4 If the output holds fewer than V vertices, **there is a cycle**.

A recursive example over a hierarchy is in `examples/sql/recursive-ctes/org_chart_tree_traversal.sql`.

[Medium] Q. What is a trie and why use one over a hash table?

Answer. A tree keyed by character, where a word's letters spell out the path from the root. Shared prefixes share nodes.

Lookup is **$O(k)$** for a key of length k - independent of how many words are stored, and with no hashing and no collisions.

The reason to choose it over a hash map is **prefix queries**. "All words starting with `pre`" is one walk to the `pre` node followed by a subtree traversal. A hash map cannot do this at all without scanning every key, because hashing deliberately destroys the relationship between similar keys.

That makes tries the structure behind autocomplete, spell-checkers and IP routing tables. The cost is memory: a node per character per distinct prefix.

Tries & Prefix Data Structures

A trie (prefix tree) stores strings so that shared prefixes are stored once. It is the go-to structure when the question mentions "prefix", "autocomplete", or "dictionary word" - and it is the backbone of command completion, spell-checking, and routing tables.

[Easy] Q. What is a trie, and what does it buy you over a hash set of words?

Answer. A trie is a tree where each edge is labelled with a character and a path from the root spells a string. What it buys you is **prefix awareness**: `search`, `insert`, and `starts_with` all cost $O(L)$ in the string length L (not the number of words), and you can enumerate every word with a given prefix by walking the subtree - something a hash set cannot do at all.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def _walk(self, s: str):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.is_end

    def starts_with(self, prefix: str) -> bool:
        return self._walk(prefix) is not None

t = Trie()
for w in ["cat", "car", "card", "care"]:
    t.insert(w)
assert t.search("cat") and t.search("card") and not t.search("ca")
assert t.starts_with("car") and not t.starts_with("dog")
```

The key detail: `search("ca")` is `False` even though `starts_with("ca")` is `True`, because `ca` is a prefix but was never itself inserted - the `is_end` flag distinguishes "a word ends here" from "a path passes through here".

[Medium] Q. How do you list all words that share a prefix, and what's the cost?

Answer. Walk to the node for the prefix ($O(L)$), then a DFS of that subtree collecting every `is_end` node. The cost is $O(L + K)$ where K is the total number of characters in all matching words - you pay for the output, which is unavoidable.

```
def words_with_prefix(self, prefix: str) -> list[str]:
    node = self._walk(prefix)
    if node is None:
```

```

        return []
    out = []
    def dfs(n, acc):
        if n.is_end:
            out.append(prefix + acc)
        for ch in sorted(n.children):
            dfs(n.children[ch], acc + ch)
    dfs(node, "")
    return out

t.words_with_prefix("ca")    # ['car', 'card', 'care', 'cat']
t.words_with_prefix("do")    # []

```

This is exactly the autocomplete step. Note the `sorted(...)` makes the output deterministic (alphabetical); drop it if you only care about existence and want to stop early.

[Medium] Q. Trie vs sorted array + binary search vs hash set - when is each right?

Answer.

Need	Best fit	Why
"Is this exact word present?"	Hash set	O(L) expected, tiny constant
"Any word with this prefix?"	Trie	Subtree walk; the other two can't
"Nearest word / suggest close match"	Trie	DFS with a small edit budget
Memory is tight, no prefix queries	Sorted array	No per-node object overhead

A hash set wins on plain membership and uses far less memory. A trie's cost is one node object per distinct prefix, so a dictionary of 100k long words can be memory-heavy - which is why production systems often use a **radix trie** (compress unary chains into a single labelled edge) or a sorted array with binary search when they don't actually need prefix enumeration.

[Hard] Q. How would you implement "longest word in the dict that is a

prefix of `s`", and what's the complexity?

Answer. Insert every dictionary word into a trie, then walk `s` character by character, tracking the deepest `is_end` node you pass. When the walk falls off a branch, stop - you've found the longest prefix that exists. Time is $O(L + \text{total dict characters})$ to build plus $O(L)$ to query; space $O(\text{total dict characters})$.

```

def longest_prefix(trie: "Trie", s: str) -> str:
    node = trie.root
    best = ""
    for i, ch in enumerate(s):
        if ch not in node.children:
            break
        node = node.children[ch]
        if node.is_end:
            best = s[: i + 1]
    return best

t = Trie()
for w in ["we", "word", "flower", "flowering"]:
    t.insert(w)
assert longest_prefix(t, "flowerpowers") == "flower"
assert longest_prefix(t, "dog") == ""

```

The trap is returning the last matched character instead of the last position where a **complete word** ended - "flowe" walks fine but isn't a word, so `is_end` is what makes the answer "flower", not "flowe".

[Easy] Q. Why do we mark a terminal node instead of, say, storing the word?

Answer. Storing the full word at each terminal node would duplicate every prefix's characters and blow the space from $O(\text{total characters})$ to $O(\text{total characters} \times \text{average length})$. A single boolean `is_end` marks the end of a word while the shared path still holds each character exactly once. If you do need the actual string (e.g. to return it), reconstruct it from the path during the walk - that's what `prefix + acc` does in the DFS above - rather than storing copies.